



Deconstructing Emotet

Executive Summary

Analysis Overview

The sample is a **384 KB Emotet loader** (**SHA-256:** `b1cad1540ecb290088252635f8e130022eed7486eb128c0ca3d676945d60a9fc`) compiled with **Visual Studio 2005**. It uses a packer to obfuscate its core logic and incorporates **anti-debugging** (`IsDebuggerPresent`) and **GUI-based anti-sandbox** techniques (`USER32/GDI32` imports).

Persistence

Upon detonation, the malware replicates itself to `C:\Windows\SysWOW64\rebrandcmp.exe`. The persistent file remains identical in hash to the original packed loader.

Core Extraction

Dynamic analysis via **x32dbg** and monitoring of `VirtualAlloc` enabled the extraction of the core payload. The unpacked binary is a **62.69 KB executable** (**SHA-256:** `96488477ed1702984c1fa3f56873e9aac4efb871f97a03eea50233b13080c2f0`) compiled with **Visual Studio 2013**.

Technical Indicators

- **Original Entry Point (OEP):** `0x0000CD97`
- **Payload Entropy:** High entropy in `.text` (6.75) and `.data` (7.162).
- **Payload Expansion:** The `.data` section expands from 4 KB (raw) to 16 KB (virtual), indicating encrypted configuration buffers.

Sample Metadata

Property	Details / Value
MD5	<code>f3f48c57c38bff2ddd220f20569e1ee6</code>
SHA-1	<code>0421127f1bcca91a6ab2a570a47f8159101b751a</code>
SHA-256	<code>b1cad1540ecb290088252635f8e130022eed7486eb128c0ca3d676945d60a9fc</code>

Property	Details / Value
Vhash	035056555d1d5517z70057mz17ez1
Authentihash	a3a64d72a568e549fa5f437b371f418445ee04c6ff6636a24318a543c23398f3
Imphash	efe1c3568d5733ccb1e9d2b524c47cea
Rich PE header hash	24ae9e717eb5d67a8cc4a0b4b12ce081
SSDEEP	3072:iYyIxN7LMWf+GPBLi21oc02jytUkU4uDQUiysA+30Sor6KH7j0m43ayYZt:GIX5MKQUJkqDDj+xW6KH7IuN
TLSH	T1A884B00532A4D4B2D89701BF8D03C33556B6F0A45B269BC377D40D9E9BA46E1BA3B3C9
File type	Win32 EXE (executable, 32-bit, GUI, win32, pe, peexe)
Magic	PE32 executable (GUI) Intel 80386, for MS Windows
TrID	Win32 Executable MS Visual C++ (generic) (47.3%), Win64 Executable (generic) (15.9%), Win32 Dynamic Link Library (generic) (9.9%), Win16 NE executable (generic) (7.6%), Win32 Executable (generic) (6.8%)
DetectItEasy	PE32; Compiler: EP:Microsoft Visual C/C++ (2005) [EXE32]; Compiler: Microsoft Visual C/C++ (14.00.50727) [C++/book]; Linker: Microsoft Linker (8.00.50727); Tool: Visual Studio (2005)
Magika	PEBIN
File size	384.00 KB (393,216 bytes)
Signature	Microsoft Linker 8.0
Entry Point Location	0x0000E022 (section [.text])
First 32 Bytes (Hex)	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
First 32 Bytes (Text)	MZ.....@.....
Entry Point Bytes (Hex)	E8 59 9C 00 00 E9 16 FE FF FF CC CC CC CC 8B 4C 24 04 F7 C1 03 00 00 00 74 24 8A 01 83 C1 01 84
Compiler Stamp	Sun Oct 06 18:38:52 2019 (UTC)
Debug Stamp	Sun Oct 06 18:38:52 2019 (UTC)
Export Stamp	Sun Oct 06 18:38:52 2019 (UTC)
Original File Name	Emetim.exe
Version/Manifest	n/a

PE Sections

Feature	.text (Section 0)	.rdata (Section 1)	.data (Section 2)	.idata (Section 3)	.rsrc (Section 4)
Entropy	5.740	5.337	3.112	4.678	0.725
File Ratio	60.42 %	34.38 %	2.08 %	1.04 %	1.04 %
Raw Address	0x00001000	0x0003B000	0x0005C000	0x0005E000	0x0005F000
Raw Size	237,568 bytes	135,168 bytes	8,192 bytes	4,096 bytes	4,096 bytes
Virtual Address	0x00001000	0x0003B000	0x0005C000	0x00061000	0x00062000
Virtual Size	236,815 bytes	132,766 bytes	16,432 bytes	4,044 bytes	590 bytes

Characteristics	0x60000020	0x40000040	0xC0000040	0xC0000040	0x40000040
Permissions	Read / Execute	Read	Read / Write	Read / Write	Read

Import Table

Capability	Associated Imports (Summary)	Source DLL
Memory Management	VirtualAlloc , VirtualFree , HeapAlloc , HeapFree , HeapReAlloc , HeapCreate , HeapDestroy	KERNEL32.dll
Process/Thread Control	GetCurrentProcess , TerminateProcess , ExitProcess , GetCurrentThreadId , Sleep , TlsAlloc/Free , RtlUnwind	KERNEL32.dll
Anti-Analysis	IsDebuggerPresent , UnhandledExceptionFilter , SetUnhandledExceptionFilter , QueryPerformanceCounter , GetTickCount	KERNEL32.dll
File / I/O Operations	CreateFileA , ReadFile , WriteFile , CloseHandle , SetFilePointer , FlushFileBuffers , GetStdHandle , SetStdHandle	KERNEL32.dll
Dynamic Loading	GetProcAddress , LoadLibraryA , FreeLibrary , GetModuleHandleA , GetModuleFileNameA	KERNEL32.dll
GUI & User Interaction	CreateWindowExA , RegisterClassExA , ShowWindow , UpdateWindow , GetMessageA , DispatchMessageA , DefWindowProcA	USER32.dll
Graphics Rendering	BitBlt , CreateCompatibleDC , SelectObject , DeleteDC , GetObjectA , DeleteObject , GetStockObject	GDI32.dll
Localization/Strings	GetLocaleInfoA/W , GetCPInfo , WideCharToMultiByte , MultiByteToWideChar , GetStringTypeA/W , LCMultiByteToWideChar	KERNEL32.dll
System Info/Env	GetVersionExA , GetCommandLineA , GetEnvironmentStrings , SetEnvironmentVariableA , GetTimeZoneInformation	KERNEL32.dll

Description

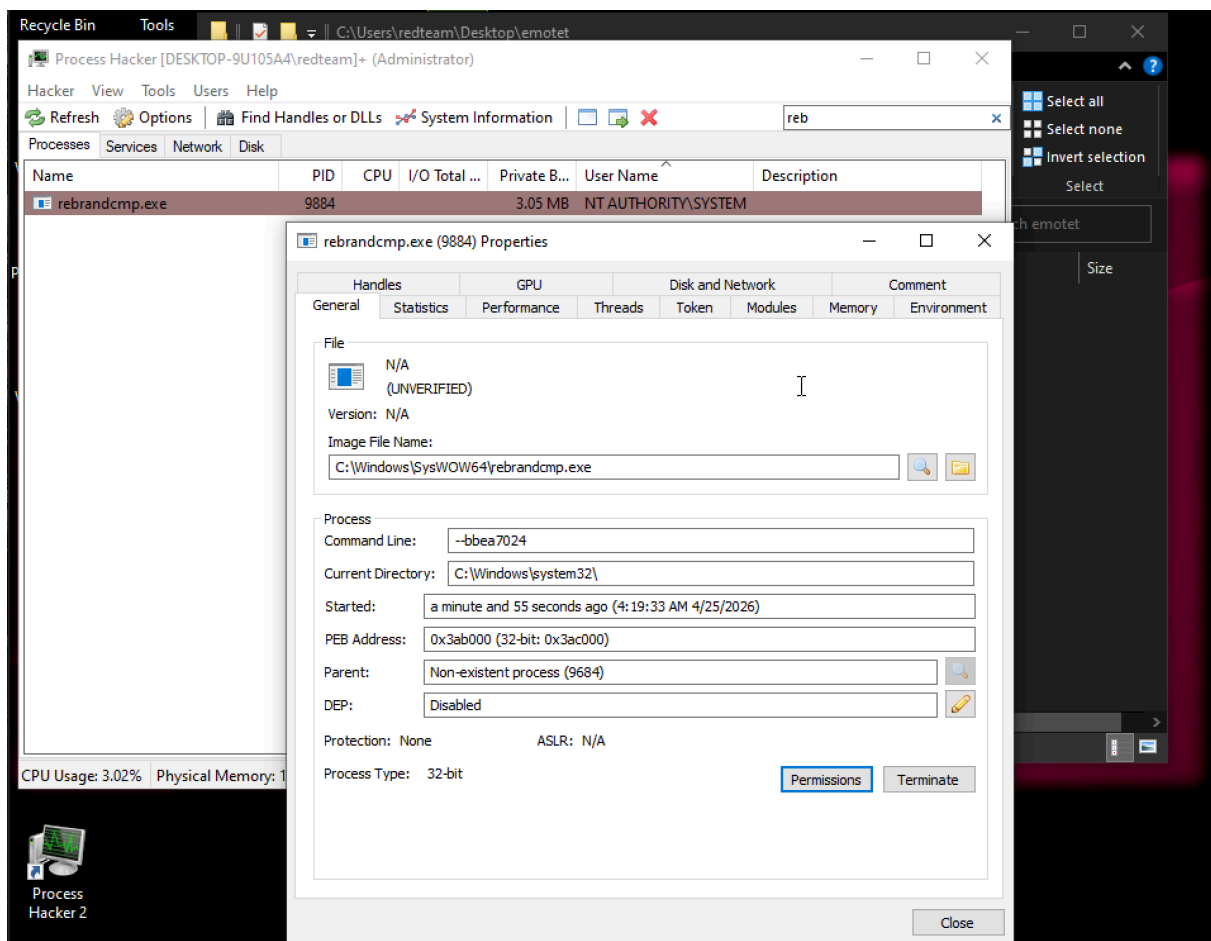
This is a **32-bit Win32 GUI executable** (384 KB) identified as an **Emotet loader**, compiled via **Visual Studio 2005** with an entry point at `0x0000E022`. Key technical indicators include:

- **Memory & Payload:** Large disparity in the `.data` section's virtual vs. raw size and high entropy in `.text`, suggesting memory-resident payload decompression.
- **Capabilities:** Imports confirm **memory manipulation** (`VirtualAlloc`), **anti-analysis** (`IsDebuggerPresent`), and **dynamic loading** (`LoadLibraryA`).

- **Evasion:** Extensive **USER32/GDI32** imports indicate GUI-based anti-sandbox techniques.
- **Metadata:** References an original filename `Emetim.exe` and debug path `Emetim.pdb`, with 2019 timestamps.

Detonation

Upon detonation, the malware initiates a persistence routine by replicating its own binary. It copies itself to the system directory at `C:\Windows\SysWOW64\rebrandcmp.exe`. The file is renamed to blend in with legitimate system components. Comparison of the SHA-256 hashes confirms that the persistence file is an identical copy of the original packed loader.



```
c:\Windows\SysWOW64
λ md5sum.exe rebrandcmp.exe
f3f48c57c38bff2ddd220f20569e1ee6 *rebrandcmp.exe
```

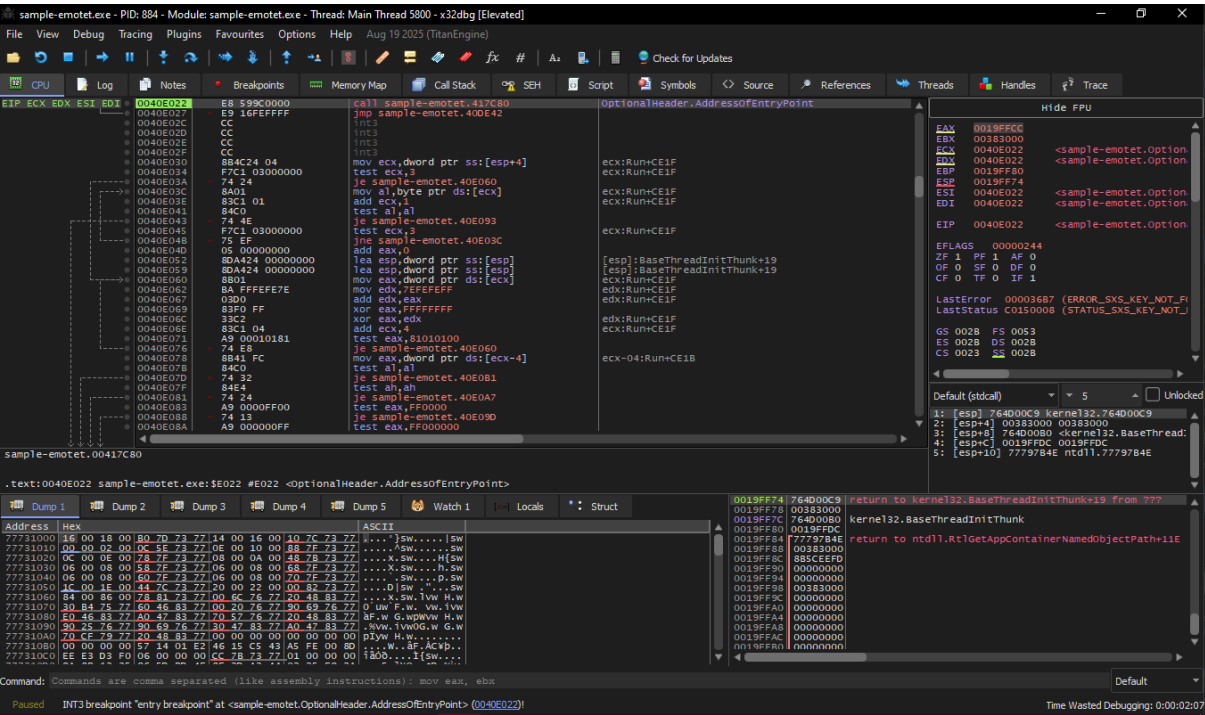
Phase	Activity	Details
Persistence	File System Replication	The sample copies itself to a system directory to ensure execution after reboot.
Source Path	<code>sample-emotet.exe</code>	Original execution path.

Target Path	C:\Windows\SysWOW64\rebrandcmp.exe	32-bit System directory (Masquerading).
File Integrity	Identical Hash	Verified via MD5; no changes made to the binary on disk.

Payload Unpacking (x32dbg)

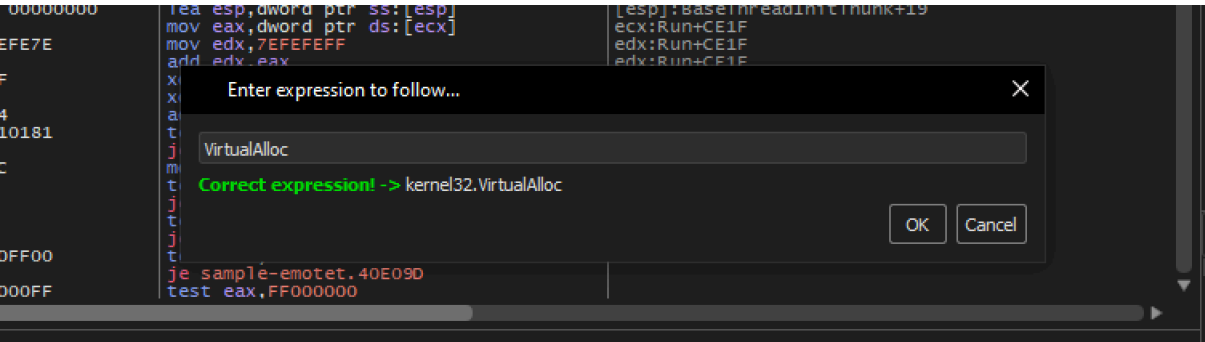
System Breakpoint

You can see the top module is set to sample-emotet.exe beside it **Entrypoint**.



One of the common techniques is to set a breakpoint on VirtualAlloc.

Now in x32dbg **ctrl + g** enter **VirtualAlloc**



We can see at the top it's set to Module Kernel32.dll, We are at the start of VirtualAlloc.

```

sample-emotet.exe - PID: 884 - Module: kernel32.dll - Thread: Main Thread 5800 - x32dbg [Elevated]
File View Debug Tracing Plugins Favourites Options Help Aug 19 2025 (TitanEngine)
CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Th
764CF9ED CC int3
764CF9EE CC int3
764CF9EF CC int3
764CF9F0 8BFF mov edi,edi VirtualAlloc
764CF9F2 55 push ebp
764CF9F3 8BEC mov ebp,esp
764CF9F5 5D pop ebp
764CF9F6 FF25 6C135376 jmp dword ptr ds:[<VirtualAlloc>] JMP.&VirtualAlloc
764CF9FD CC int3
764CF9FE CC int3
764CF9FF CC int3
764CFA00 CC int3
764CFA01 CC int3
764CFA02 CC int3
764CFA03 CC int3
764CFA04 CC int3
764CFA05 CC int3
764CFA06 CC int3
764CFA07 CC int3
764CFA08 CC int3
764CFA09 CC int3
764CFA0A CC int3
764CFA0B CC int3
764CFA0C CC int3
764CFA0D CC int3
764CFA0E CC int3
764CFA0F CC int3
764CFA10 8BFF mov edi,edi MoveFileA
764CFA12 55 push ebp
764CFA13 8BEC mov ebp,esp
764CFA15 8B55 0C mov edx,dword ptr ss:[ebp+C] edx:Run+CE1F
764CFA18 33C0 xor eax,eax ecx:Run+CE1F

```

The Return Address Strategy

When `VirtualAlloc` is called, the OS hasn't actually given the malware the memory address yet. You need to see where that memory lands.

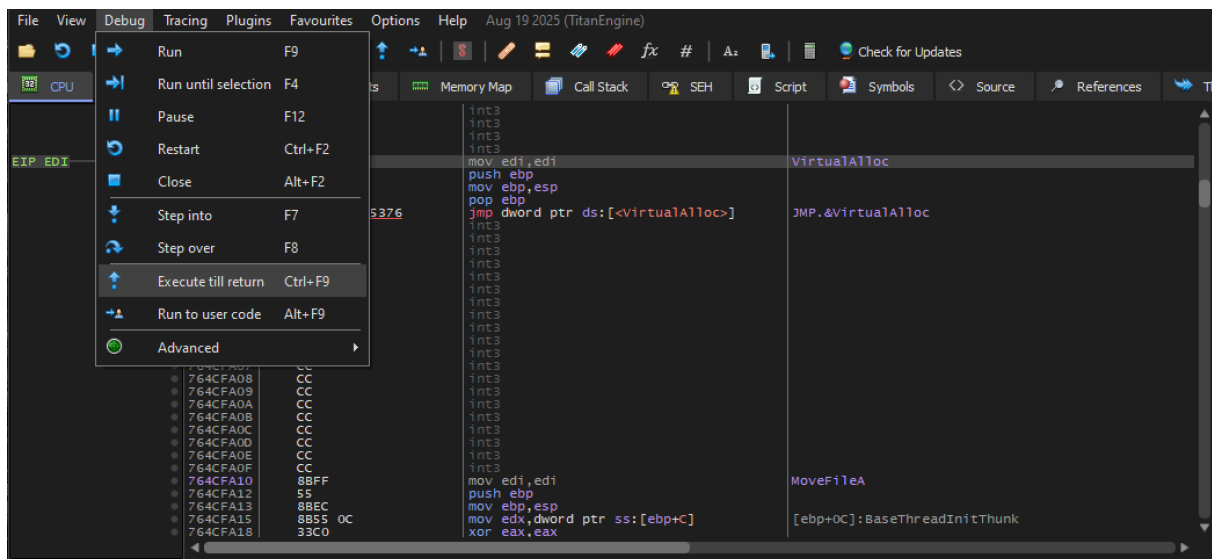
Set Breakpoint. then run it.

```

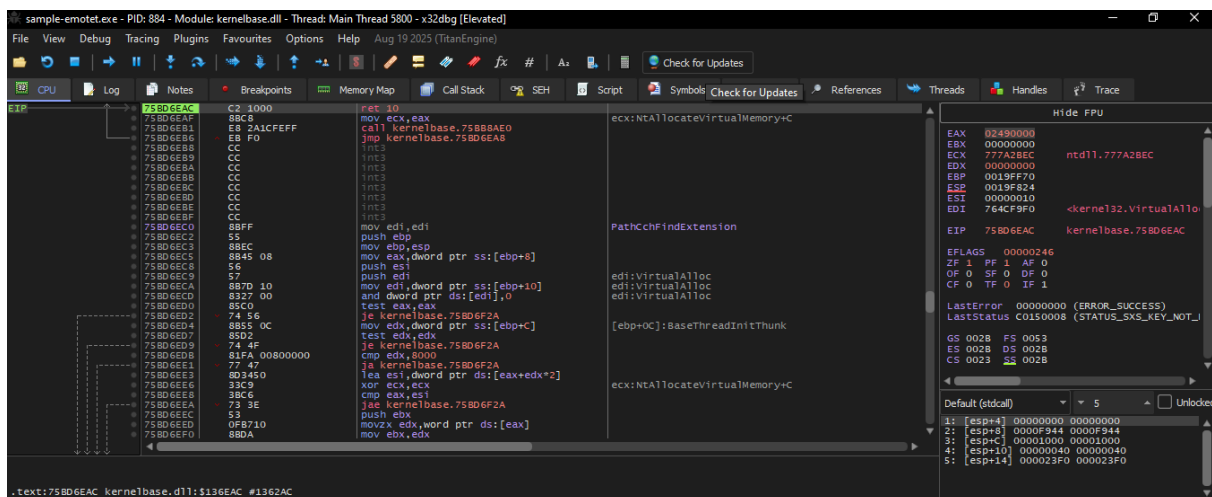
sample-emotet.exe - PID: 884 - Module: kernel32.dll - Thread: Main Thread 5800 - x32dbg [Elevated]
File View Debug Tracing Plugins Favourites Options Help Aug 19 2025 (TitanEngine)
CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Th
764CF9ED CC int3
764CF9EE CC int3
764CF9EF CC int3
764CF9F0 8BFF mov edi,edi VirtualAlloc
764CF9F2 55 push ebp
764CF9F3 8BEC mov ebp,esp
764CF9F5 5D pop ebp
764CF9F6 FF25 6C135376 jmp dword ptr ds:[<VirtualAlloc>] JMP.&VirtualAlloc
764CF9FD CC int3
764CF9FE CC int3
764CF9FF CC int3
764CFA00 CC int3
764CFA01 CC int3
764CFA02 CC int3
764CFA03 CC int3
764CFA04 CC int3
764CFA05 CC int3
764CFA06 CC int3
764CFA07 CC int3
764CFA08 CC int3
764CFA09 CC int3
764CFA0A CC int3
764CFA0B CC int3
764CFA0C CC int3
764CFA0D CC int3
764CFA0E CC int3
764CFA0F CC int3
764CFA10 8BFF mov edi,edi MoveFileA
764CFA12 55 push ebp
764CFA13 8BEC mov ebp,esp
764CFA15 8B55 0C mov edx,dword ptr ss:[ebp+C] edx:Run+CE1F
764CFA18 33C0 xor eax,eax

```

Break point hit, now execute till return.

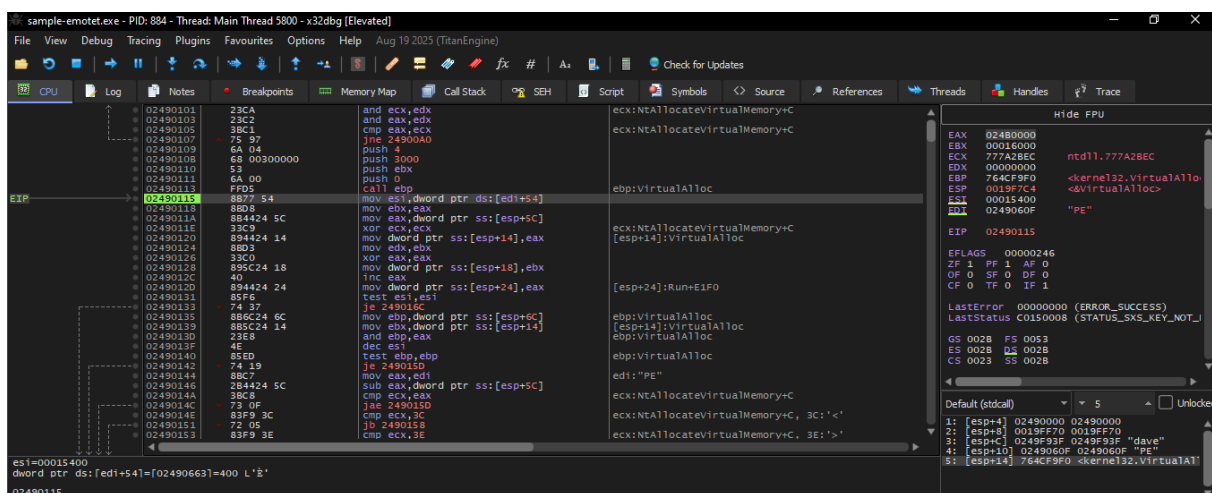


Reached return address



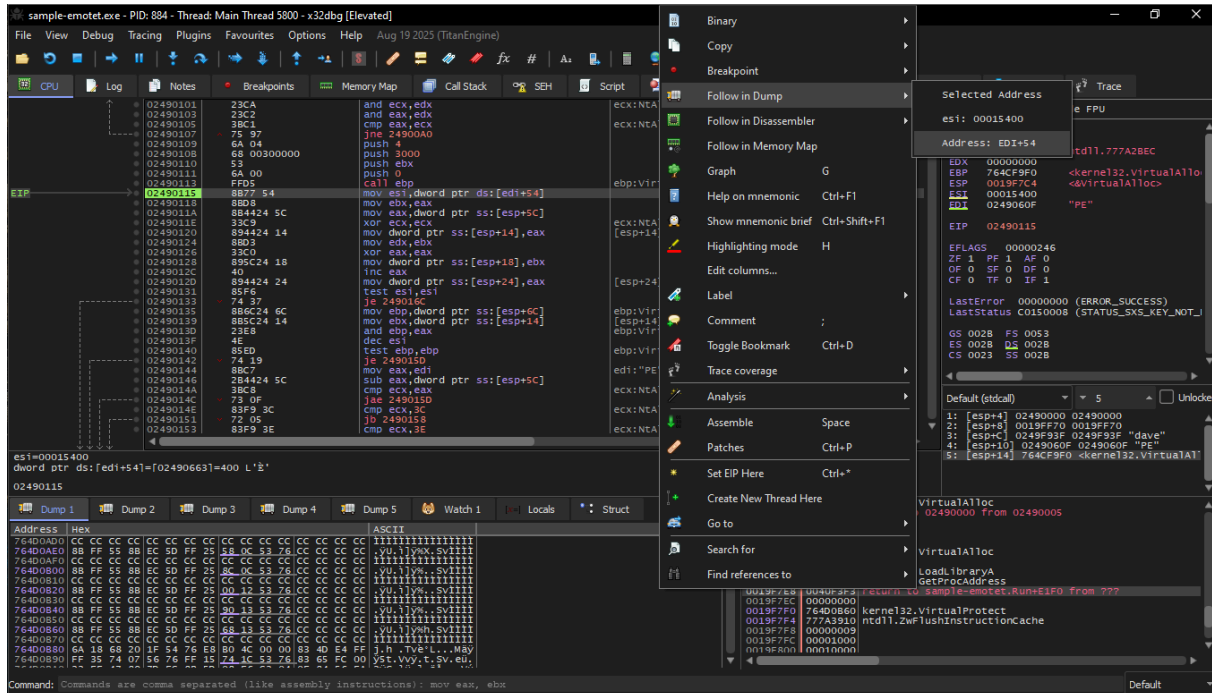
Now step over press: **F8**

We can see a call to **ebp** . That's where **VirtualAlloc** been stored.

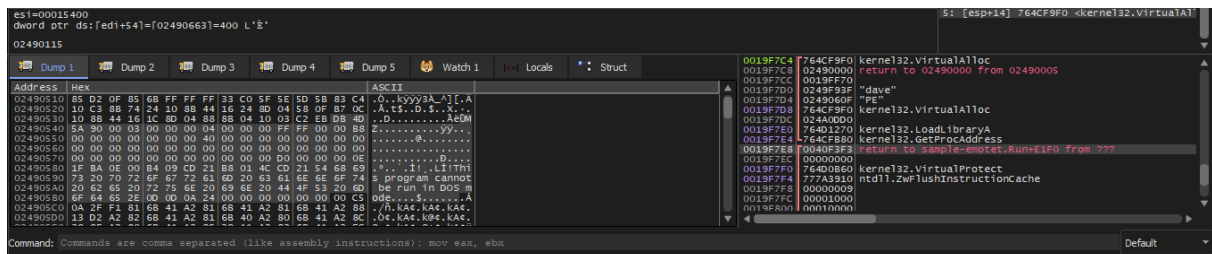


Follow in Dump

follow the argument `[edi+54]` .



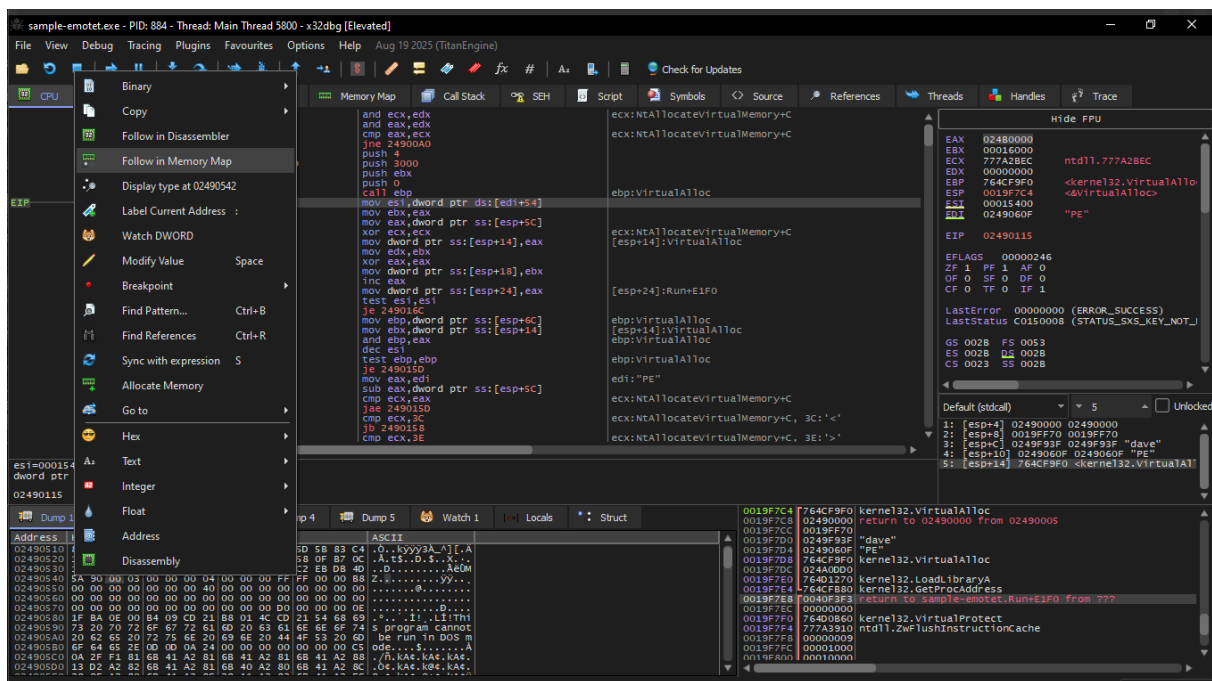
In the memory dump scroll down a bit look for `mz header` .



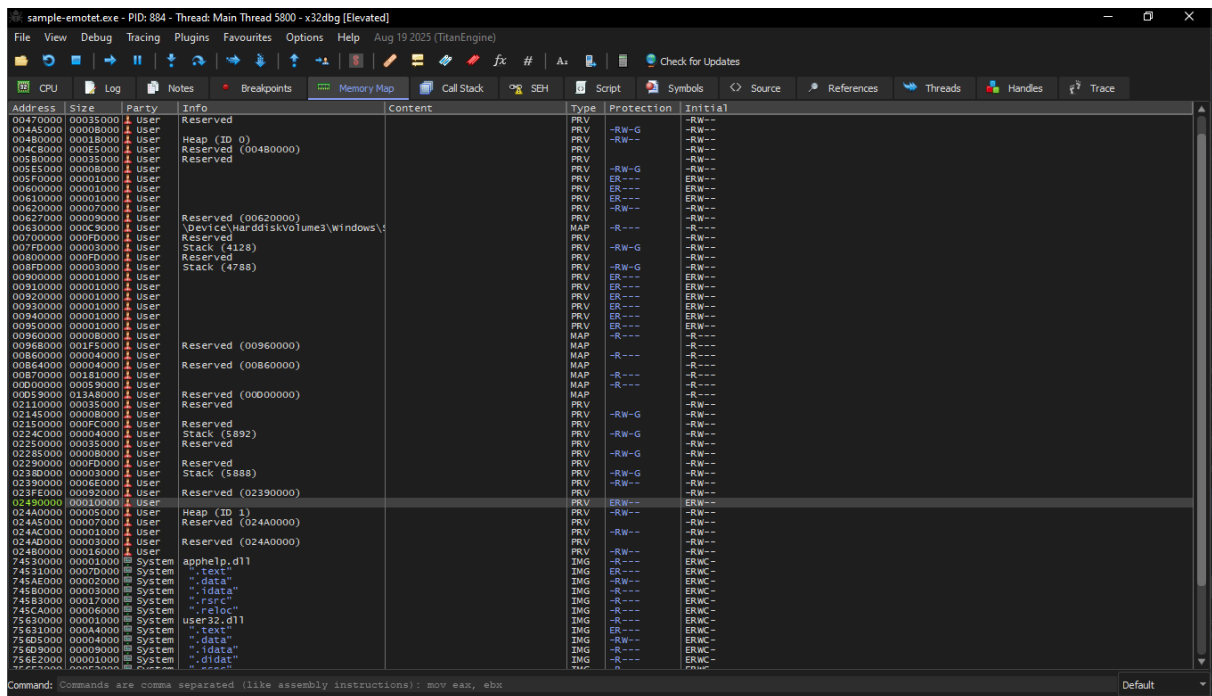
So we found a executable here which could potentially be our unpacked code.

Follow in Memory Map

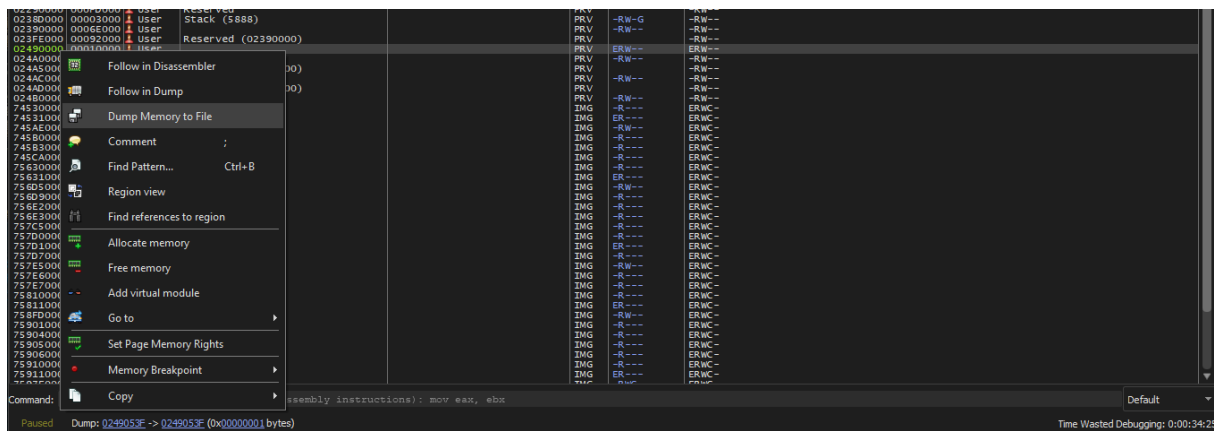
Right click in the memory map space then follow in memory map.



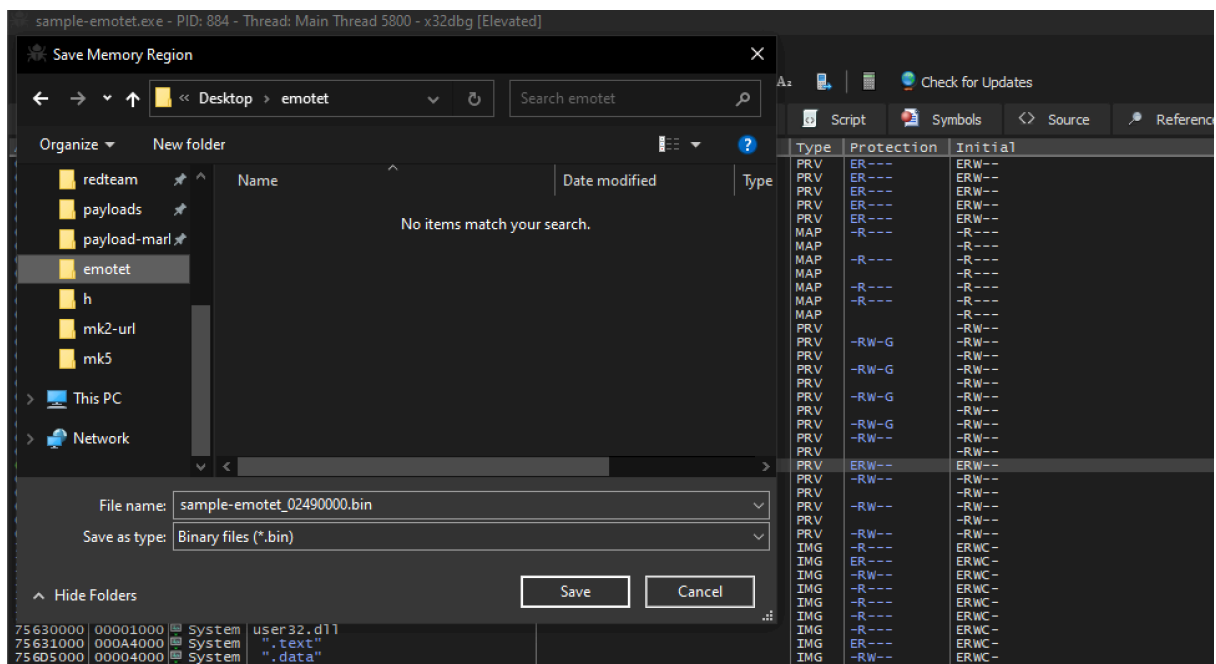
Inspecting the **Protection** column in the **Memory Map** tab is one of the most reliable ways to narrow down where the "real" malware is hiding.



Right click on the memory address then Dump Memory to File.



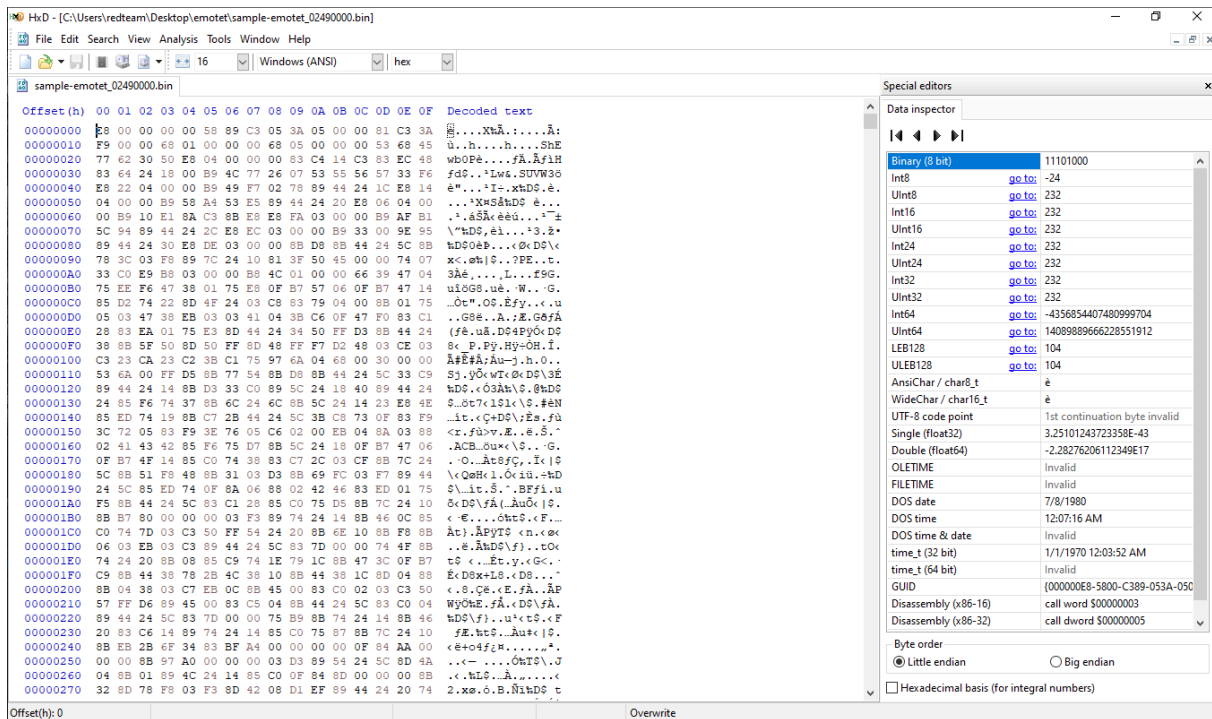
Save it



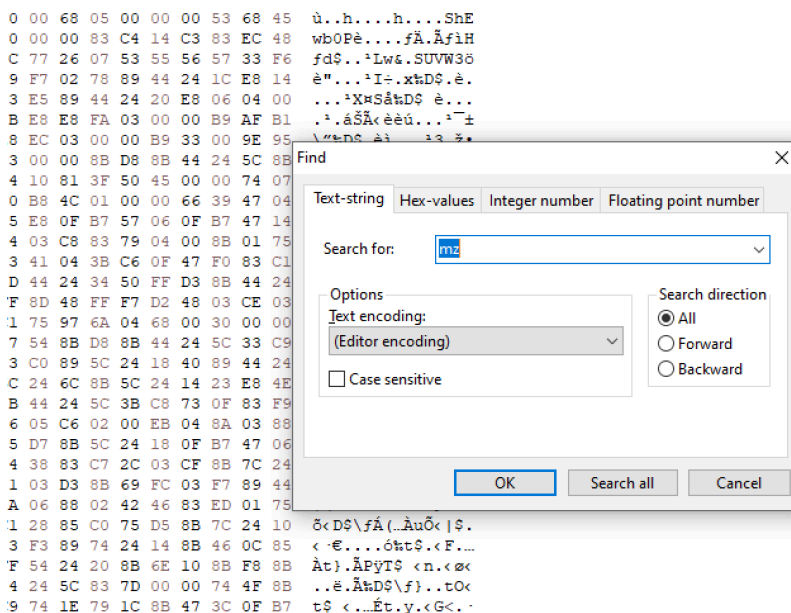
Hxd

Opening the dumped file in hxd.

We don't actually have a clean executable, there's bunch of junk code.



Search for mz



Binary (8 bit)	01001101
Int8	77
UInt8	77
Int16	23117
UInt16	23117
Int24	Invalid
UInt24	Invalid
Int32	Invalid
UInt32	Invalid
Int64	Invalid
UInt64	Invalid
LEB128	-51
ULEB128	77
AnsiChar / char8_t	M
WideChar / char16_t	M
UTF-8 code point	M (U+00
Single (float32)	Invalid
Double (float64)	Invalid
OLETIME	Invalid
FILETIME	Invalid
DOS date	2/13/202
DOS time	11:18:26
DOS time & date	Invalid
time_t (32 bit)	Invalid
time_t (64 bit)	Invalid

Select the above junk code and delete it. save it for clean PE file.

Feature	.text	.rdata	.data	.CRT	.reloc
Permissions	Read/Execute	Read	Read/Write	Read	Read/Discard

Imports

KERNEL32.dll

- IsProcessorFeaturePresent